CORE JAVA MINI PROJECT

Bank Management System

OOPs + Exception Handling Project Guide

Purpose: This guide is designed so students can read the requirements, understand the class responsibilities, identify the OOP concepts and exceptions required, and build the project themselves. It intentionally does not provide the complete project solution.

1. Project Overview

Build a console-based Bank Management System using Core Java. The project must focus only on Object-Oriented Programming and Exception Handling. Students should not use Collections, JDBC, File Handling, Stream API, Lambda expressions, or a database.

Main objective: Convert real-world banking requirements into classes and objects, use inheritance and abstraction to model account types, use runtime polymorphism for account-specific behavior, and use custom exceptions to handle invalid operations safely.

2. Concepts to Be Used

Category	Concepts
OOP	Class, Object, Constructor, this, super, Encapsulation
Inheritance	extends, parent-child relationship
Abstraction	abstract class, abstract method
Polymorphism	Parent reference referring to child object, method overriding
Exception Handling	try, catch, finally, throw, throws
Custom Exceptions	InvalidAmountException, InsufficientBalanceException, AccountNotFoundException

3. Project Features

• Create Savings Account

• Create Current Account

• Display Account Details

• Deposit Money

• Withdraw Money

• Check Balance

• Transfer Money

• Calculate Interest

• Handle invalid amounts

• Handle insufficient balance

• Handle non-existing accounts

• Exit without crashing the application

4. Application Menu

```
================================
BANK MANAGEMENT SYSTEM
================================

1. Create Savings Account
2. Create Current Account
3. Display Account
4. Deposit
5. Withdraw
6. Transfer Money
7. Check Balance
8. Calculate Interest
9. Exit

Enter Choice:
```

5. Suggested Class Design

```
Account
|
-------------------
| |
SavingsAccount CurrentAccount

Bank
|
+-- manages accounts and banking operations

Custom Exceptions
|
+-- InvalidAmountException
+-- InsufficientBalanceException
+-- AccountNotFoundException
```

The exact class design may be improved by the student, but the project must clearly separate account data, account behavior, banking operations, and exception types.

6. Account – Base Class

Create an **abstract class Account**. It represents common properties and operations shared by all account types.

```
abstract class Account {

private int accountNumber;
private String holderName;
private double balance;

// constructor
// getters and setters

// deposit()
// withdraw()
// display()

abstract void calculateInterest();
}
```

Why abstract? Account is a general concept. The interest calculation can be different for Savings and Current accounts, so the parent class defines the method while child classes provide the implementation.

7. Encapsulation

The account fields must be private. Access them through public getters and setters or controlled methods.

```
private int accountNumber;
private String holderName;
private double balance;
```

Students should be able to explain: **Encapsulation means wrapping data and methods together and controlling direct access to the data.**

8. SavingsAccount

Create **SavingsAccount extends Account**. Use an interest rate of 4% for this project. Override calculateInterest().

```
class SavingsAccount extends Account {

@Override
void calculateInterest() {
// calculate 4% interest
}
}
```

The student must decide how to display the interest and should explain why overriding is required.

9. CurrentAccount

Create **CurrentAccount extends Account**. For this project, use 0% interest. You may additionally enforce a minimum balance of 5000 as an extension.

```
class CurrentAccount extends Account {

@Override
void calculateInterest() {
// current account interest logic
}
}
```

10. Runtime Polymorphism

Demonstrate runtime polymorphism using an Account reference.

```
Account account;

account = new SavingsAccount();
account.calculateInterest();

account = new CurrentAccount();
account.calculateInterest();
```

The same parent reference calls different child implementations. Students must explain why the method executed depends on the actual object created at runtime.

11. Deposit Operation

Required behavior:

• Ask for account number.

• Find the account.

• Ask for deposit amount.

• Reject zero or negative amounts.

• Add the amount to balance.

• Display the updated balance.

```
Enter Account Number: 101
Enter Amount: 5000

Amount deposited successfully.
Current Balance: 15000
```

If amount is less than or equal to zero, throw InvalidAmountException.

12. Withdraw Operation

Required behavior:

• Ask for account number.

• Find the account.

• Ask for withdrawal amount.

• Reject zero or negative amounts.

• Check available balance.

• Withdraw only if sufficient balance exists.

```
Enter Account Number: 101
Enter Amount: 12000

Insufficient balance.
Available Balance: 10000
```

If withdrawal amount is greater than balance, throw InsufficientBalanceException.

13. Transfer Money

Transfer money from one account to another.

```
From Account : 101
To Account : 102
Amount : 5000

Transfer Successful
```

The operation must validate both accounts and the amount, then verify the sender has enough balance. This feature is useful for demonstrating multiple possible exceptions in one operation.

14. Custom Exceptions

Create checked custom exceptions by extending Exception.

```
class InvalidAmountException extends Exception {
public InvalidAmountException(String message) {
super(message);
}
```

```
}

class InsufficientBalanceException extends Exception {
public InsufficientBalanceException(String message) {
super(message);
}
}

class AccountNotFoundException extends Exception {
public AccountNotFoundException(String message) {
super(message);
}
}
```

Students should understand that custom exceptions allow the application to report business-specific problems clearly instead of using generic messages.

15. Exception Handling Requirements

• Use try-catch around operations that may fail.

• Use finally at least once to demonstrate cleanup/final execution.

• Use throw when the program detects an invalid business condition.

• Use throws in method declarations where checked exceptions are propagated.

• Use custom exceptions for account and banking validation.

• The program must not terminate unexpectedly for normal invalid input.

```
try {
// banking operation
}
catch (InvalidAmountException e) {
System.out.println(e.getMessage());
}
catch (InsufficientBalanceException e) {
System.out.println(e.getMessage());
}
finally {
System.out.println("Operation completed.");
}
```

16. Bank Class / Service Responsibility

Create a class such as **Bank** or **BankService** to manage banking operations. It should be responsible for creating accounts, finding accounts, depositing, withdrawing, transferring, displaying details, and calculating interest.

For this project version, students may use a fixed-size array of Account references. Do not use ArrayList or other Collections.

```
private Account[] accounts;
```

The student must decide the maximum number of accounts and how to track the current count.

17. Account Search

Create a method that searches the account storage using account number. If no matching account exists, throw AccountNotFoundException.

```
findAccount(int accountNumber)
-> return matching Account
-> otherwise throw AccountNotFoundException
```

18. Account Creation

Savings account example:

```
Enter Account Number: 101
Enter Holder Name: Rahul
Enter Initial Balance: 10000

Savings Account created successfully.
```

Current account example:

```
Enter Account Number: 102
Enter Holder Name: Priya
Enter Initial Balance: 15000

Current Account created successfully.
```

The student should prevent duplicate account numbers and reject invalid initial balances.

19. Display Account

```
Account Number : 101
Holder Name : Rahul
Account Type : Savings
Balance : 10000
Interest : 400
```

Students may use method overriding to display account-specific information if they want an additional polymorphism exercise.

20. Check Balance

```
Enter Account Number: 101

Current Balance: 10000
```

If the account does not exist, AccountNotFoundException must be handled.

21. Project Flow

```
START
|
v
Create Bank object
|
v
Display Menu
|
v
Read Choice
|
+--> Create Savings Account
|
+--> Create Current Account
|
+--> Display Account
|
```

```
+--> Deposit
|
+--> Withdraw
|
+--> Transfer
|
+--> Check Balance
|
+--> Calculate Interest
|
+--> Exit
|
v
Handle exceptions
|
v
Continue menu until Exit
```

22. Required Invalid Scenarios

• Deposit -500

• Withdraw -100

• Deposit 0

• Withdraw 0

• Withdraw more than available balance

• Search an account that does not exist

• Transfer to an account that does not exist

• Transfer a negative amount

• Transfer more money than sender balance

• Create duplicate account number

• Enter an invalid menu option

For each scenario, display a meaningful message and continue the application.

23. Recommended Development Order

Step 1: Create Account abstract class and fields.

Step 2: Add constructor, getters, setters and common methods.

Step 3: Create SavingsAccount and CurrentAccount.

Step 4: Override calculateInterest().

Step 5: Create Bank/BankService class.

Step 6: Implement account creation.

Step 7: Implement account search.

Step 8: Implement deposit and withdraw.

Step 9: Create and use custom exceptions.

Step 10: Implement transfer.

Step 11: Implement display, balance and interest operations.

Step 12: Build the menu using switch and exception handling.

Step 13: Test every invalid scenario.

24. Restrictions

• No ArrayList, HashMap, HashSet or other Collections.

• No JDBC.

• No database.

• No File Handling.

• No Stream API.

• No Lambda expressions.

• No Spring or other frameworks.

• Focus only on Core Java OOPs and Exception Handling.

25. Student Submission Requirements

• Complete Java source code.

• All classes properly separated.

• Custom exception classes.

• Meaningful class, method and variable names.

• At least 10 valid test cases.

• At least 10 invalid/exception test cases.

• Short project explanation.

• Class diagram or hand-drawn class relationship diagram.

• Screenshots of important outputs.

26. Viva / Interview Questions

1. Why is Account an abstract class?

2. Why are account fields private?

3. What is encapsulation in your project?

4. Where did you use inheritance?

5. Where did you use method overriding?

6. Explain runtime polymorphism using your project.

7. Why did you use an Account reference?

8. What is the purpose of super?

9. What is the purpose of this?

10. Why are custom exceptions required?

11. Difference between throw and throws?

12. Difference between checked and unchecked exceptions?

13. Why should insufficient balance be treated as an exception?

14. What happens when an account is not found?

15. Why should the application continue after an exception?

27. Final Testing Checklist

■ Create Savings Account

■ Create Current Account

■ Display account

■ Deposit valid amount

■ Deposit invalid amount

■ Withdraw valid amount

■ Withdraw more than balance

■ Search existing account

■ Search non-existing account

■ Transfer valid amount

■ Transfer invalid amount

■ Calculate Savings interest

■ Calculate Current interest

■ Verify runtime polymorphism

■ Verify custom exceptions

■ Verify try-catch-finally

■ Verify application continues after an exception

28. What Students Should Be Able to Explain

At the end of the project, every student should be able to explain the complete flow without reading the source code. They should clearly identify where Class, Object, Encapsulation, Inheritance, Abstraction, Method Overriding, Runtime Polymorphism, try-catch, finally, throw, throws, and Custom Exceptions are used.

Important: The purpose of this document is to give the project idea and requirements. Students should write the implementation themselves. Do not copy a complete solution without understanding each class and operation.

29. Suggested Project Title

Core Java Mini Project – Bank Management System
Focus: OOPs + Exception Handling
Application Type: Console Based
Level: Beginner to Intermediate